

APUNTES

Víctor Manuel Peiró Martínez
FIIS 3ºA Programación Gráfica

Contenido

Tema 1 – Introducción.....	3
La Pantalla	3
Gráficos	3
Primitivas Gráficas	4
Cauce Gráfico	5
Tema 2 – Transformaciones.....	7
Fundamentos Matemáticos	7
Matrices de Transformaciones.....	7
Modelo, Vista, Proyección.....	8
Rotaciones	8
Tema 2B – GLFW y OpenGL	10
GLFW	10
Inicialización	10
Manejo de Ventana	10
Entrada de Datos.....	11
Gestión de Tiempo.....	11
OpenGL	12
Estados.....	12
Buffers	12
Pintado.....	13
Tema 3 – Técnicas de Representación y Modelado.....	15
Introducción.....	15
Técnicas de Representación de Objetos	16
Cámara.....	18
Tema 4 – Shaders y Programación en GPU	19
Shaders	19
Tema 5 – Iluminación y Texturas	24
Iluminación.....	24
Modelo de Iluminación De Phong.....	25
Sombreado.....	26
Shaders	27
Texturas	28

Mezclado de Color.....	29
Orden de Pintado	29
Tema 6 – Colisiones	31
Volúmenes Envolventes.....	31
Esferas	31
Axis Aligned Bounding Boxes (AABB)	31
Oriented Bounding Boxes (OBB).....	32
Discrete-Orientation Polytopes.....	32
Otros	32
Jerarquía de Volúmenes Envolventes	32
Árboles	33
Spatial Partition.....	34
Grids Jerárquicos	34

Tema 1 – Introducción

La Pantalla

Los gráficos se representan sobre un plano bidimensional

Se utiliza un sistema de coordenadas cartesianas

- **Eje X:** Translación sobre el eje horizontal o de abscisas
- **Eje Y:** Translación sobre el eje vertical o de ordenadas

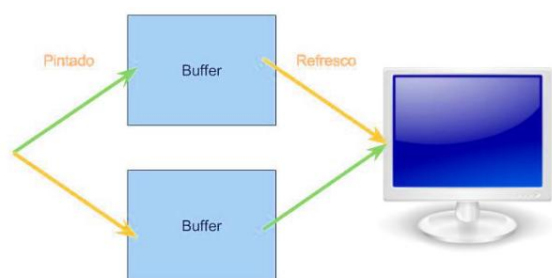
Se pueden realizar rotaciones de los gráficos y escalar los elementos

Los gráficos no se dibujan sobre pantalla sino sobre un buffer de memoria.

El contenido de la pantalla se refresca a una tasa determinada y se vuelca en la pantalla con cada refresco.

Se usa una técnica de doble buffer:

- **Back Buffer:** Buffer donde se realiza el pintado
- **Front Buffer:** Buffer donde se realiza el refresco de los datos
- **Swap Chain:** Tras cada refresco se alternan los buffers para que el ultimo buffer que se pinte será el primero que se muestre.
- **Nota:** Puede haber más de 2 buffers



Gráficos

Gráficos Rasterizados: Gráficos que tienen la información sobre los pixeles que han de ser dibujados

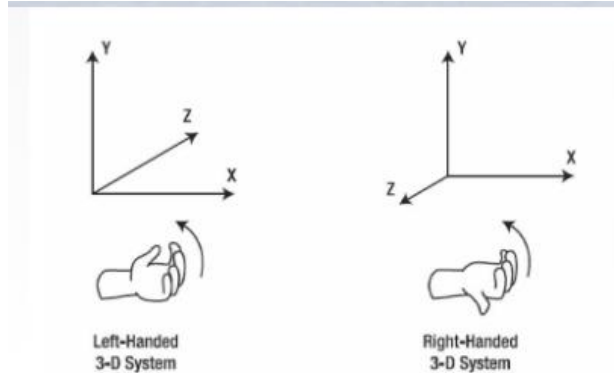
- Contienen información sobre el tamaño del bloque de pixeles a dibujar
- No contiene información sobre la forma del objeto que representan
- Si se quieren dibujar a un tamaño diferente, se hace interpolación de pixeles y se pierde nitidez
- **Ejemplos:** BMP, JPG, PNG

Gráficos Vectoriales: Gráficos que almacenan información sobre la geometría a dibujar

- Puntos que definen la geometría, líneas de unión, color/grosor de línea
- La información para dibujar debe convertirse en un gráfico rasterizado
- No se pierde nitidez

Gráficos Tridimensionales: Se representan usando espacios vectoriales

- Se representan usando gráficos vectoriales
- Las figuras geométricas se representan en el espacio euclídeo. Se añade z
- Se crean volúmenes usando primitivas gráficas
- El sistema de coordenadas puede ser mano izquierda o mano derecha



Primitivas Gráficas

Primitiva Gráfica: Elementos más simples usados para almacenar información

- A mayor complejidad, más información es presentada al usuario

Primitivas Gráficas 2D:

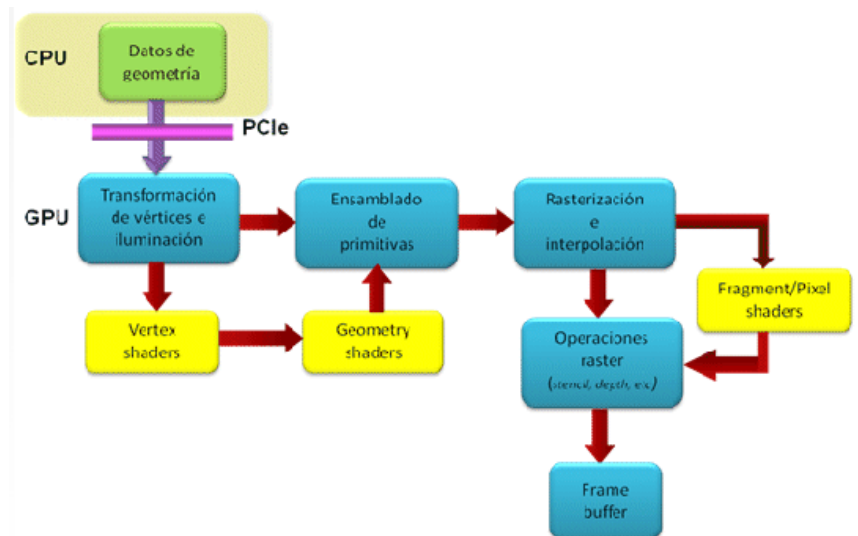
- **Píxel:** Almacena la posición (x, y) y el color de un punto en el espacio.
 - Puede agregar las transparencias
- **Agrupación de Píxeles:** Dan como resultado dibujos 2D. Pueden almacenar la capa a la que permanecen
 - **Tiles (Losas):** Permiten formar dibujos mas grandes en base a la repetición de uno o varios tiles
 - **Sprites:** Dibujos formado por un conjunto de píxeles o la unión de varios tiles
- **Capas:** Podemos formar imágenes mas complejas si colocamos las imágenes en capas

Primitivas Gráficas 3D:

- **Vértice:** Punto en el espacio que almacena las coordenadas (x, y, z), el color y opcionalmente una coordenada de textura
- **Arista:** Línea de entorno entre 2 vértices
- **Polígono:** Unión de varias aristas de forma cerrada. Cumple las siguientes condiciones:
 - Todas las aristas están en el mismo plano
 - Las aristas no intersecan entre ellas
- **Malla 3D:** Unión de varios polígonos
 - Todos los polígonos se hallan conectados

Cauce Gráfico

Cauce Gráfico: Sucesión de pasos por los que tiene que pasar una primitiva grafica desde CPU hasta que la tarjeta gráfica la muestre en el monitor



Fases:

- **Emisión de primitivas gráficas desde la CPU:**
- **Transformaciones**
- **Iluminación de vértices**
 - Se realizan operaciones sobre los vértices en función de los focos de iluminación
 - La iluminación de vértices mediante interpolación de valores
- **Proyección**
 - La escena se transforma a un espacio de coordenadas llamado espacio homogéneo de clipping
 - **Proyección Tipo Ortográfica:** Los objetos se ven del mismo tamaño sin importar la cámara
 - **Proyección Tipo Perspectiva:** A mayor distancia, más pequeños se ven los objetos
- **Clipping**
 - Se recorta la escena para que solo queden objetos que se vean desde el punto de vista de la cámara dependiendo del ángulo de visión
 - Acelera las siguientes fases. Menos primitivas = Rasterización más rápida
- **Rasterización**
 - Se transforman primitivas 3D en fragmentos o primitivas 2D
 - El cauce gráfico ahora se basa en píxeles y conserva sus atributos
- **Texturizado y sombreado**

Historia de las Tarjetas Gráficas: Las tarjetas gráficas se especializan con procesadores preparados para multiplicación de matrices y vectores

- **Primera Generación:** Gráficos formados por alambres
 - Solo coloreado de mallas y las fases de transformación
 - Clipping antes de rasterizar
- **Segunda Generación:**
 - Se añade iluminación de vértices y sombreado de polígonos
 - Se añade buffer de profundidad
- **Tercera Generación:**
 - Se añaden fases para mapear texturas sobre los polígonos
 - Transformaciones sobre coordenadas de textura
- **Cuarta Generación:**
 - Cauce Gráfico Programable. Las fases de operaciones se reparten sobre varios procesadores
 - Cada procesador puede ejecutar el programa estándar
- **Quinta Generación:**
 - Iluminación Global

Tema 2 – Transformaciones

Fundamentos Matemáticos

Coordenadas Homogéneas: Amplían el espacio euclídeo al espacio proyectivo

Representación: Tuplas de Tipo (x,y,z,w)

- Las coordenadas de espacio euclideo se hayan dividiendo cada componente respecto a w

Coordenada w: Solo puede ser 0 o 1

- **Valor 1:** Las coordenadas representan un punto del espacio
- **Valor 0:** Las coordenadas representan un vector en el espacio

Matrices de Transformaciones

Se utilizan matrices de orden 4 para representar las transformaciones en un sistema de coordenadas homogéneas

Matriz De Traslación:

$$\begin{bmatrix} 1 & 0 & 0 & X \\ 0 & 1 & 0 & Y \\ 0 & 0 & 1 & Z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Matriz de Escalado:

$$\begin{bmatrix} x & 0 & 0 & 0 \\ 0 & y & 0 & 0 \\ 0 & 0 & z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Matrices de Rotación:

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad \text{Rotación en x}$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad \text{Rotación en y}$$

$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} \quad \text{Rotación en z}$$

Modelo, Vista, Proyección

Cuando definimos los vértices del modelo, sus coordenadas son relativas al origen del modelo

Matriz Modelo: Queremos aplicar una serie de transformaciones a un modelo para situarlo en escena. Generamos una matriz con las transformaciones a aplicar y las aplicamos

- **Orden de Transformaciones:** Escalado * Rotación * Traslación

Matriz Vista: Cuando queremos visualizar la escena, lo hacemos desde la transformación relativa de la cámara. Se define matriz con la transformación del observador

- Al multiplicar los vértices por esta matriz pasamos al espacio del observador

Matriz Proyección: Aunque tengamos los vértices en el espacio del observador, tenemos que proyectarlos sobre el plano.

- **Proyección Ortogonal:** Las líneas de proyección son paralelas. El tamaño de los objetos no disminuye con la distancia
- **Proyección Con Perspectiva:** Las líneas de proyección son oblicuas causando una disminución aparente de los objetos con la distancia

Secuencia de Transformaciones:

1. Coordenadas de modelo
2. Coordenadas del mundo 3D
3. Coordenadas de la cámara (Vista)
4. Coordenadas homogéneas (Proyección)

Matriz MVP: Matriz que tiene la combinación de la matriz modelo, vista y proyección

Rotaciones

Rotación Euler: Mediante un vector (x,y,z)

- Las rotaciones se hacen sobre cada eje en torno a un eje
- **Inconvenientes:**
 - Interpolar entre 2 orientaciones es complicado
 - Siempre hay 2 rotaciones que describen la misma orientación
 - El orden de rotación alter la orientación final
 - **Gimbal Lock:** Bloqueo de rotaciones en algunas situaciones

Rotación por Cuaterniones: Rotaciones mediante un vector (x, y, z, 0) que representa el eje de rotación y un ángulo α que representa la rotación en torno a un eje

- Tupla (w, x, y, z) para representar rotaciones

- $w = \cos\left(\frac{\alpha}{2}\right)$
- $x = eje.x * \sin\left(\frac{\alpha}{2}\right)$
- $y = eje.y * \sin\left(\frac{\alpha}{2}\right)$
- $z = eje.z * \sin\left(\frac{\alpha}{2}\right)$

```
eje.x = x / sqrt(x2 + y2 + z2)
eje.y = y / sqrt(x2 + y2 + z2)
eje.z = z / sqrt(x2 + y2 + z2)
angulo = acos(w) * 2
```

- **Matriz De Rotación:** Los vectores han de estar normalizados

$$\begin{pmatrix} (1 - 2*y*y - 2*z*z), & (2*x*y - 2*w*z) & , & (2*x*z + 2*w*y) & , & 0, \\ (2*x*y + 2*w*z) & , & (1 - 2*x*x - 2*z*z), & (2*y*z - 2*w*x) & , & 0, \\ (2*x*z - 2*w*y) & , & (2*y*z + 2*w*x) & , & (1 - 2*x*x - 2*y*y), & 0, \\ 0, & & 0, & & 0, & 1); \end{pmatrix}$$

- **Ventajas:**
 - No sufre gimbal lock
 - Interpolación sencilla
 - Menos errores de redondeo
 - Operaciones son más rápidas
- **Desventajas:**
 - Es más difícil visualizar una rotación

Tema 2B – GLFW y OpenGL

GLFW

Inicialización

glfwInit(): Sirve para inicializar la librería. Devuelve GLFW_TRUE si se ha podido inicializar o GLFW_FALSE en el caso contrario

glfwTerminate(): Permite desalojar todos los recursos utilizados por GLFW

#define GLAD_BIN: Define cargador de OpenGL

#include “common.h”: Este archivo es el que tiene la librería preparada para uso. Tiene que ir en el main 100%

gladLoadGL(glfwGetProcAddress): Obtiene y carga todas las funciones de OpenGL

- **gladLoadGL():** Inicializa GLAD cargando los punteros a las funciones de OpenGL
- **glfwGetProcAddress:** Devuelve la dirección de memoria de las funciones de OpenGL

Manejo de Ventana

GLFWwindow* glfwCreateWindow(): Crea la ventana

- **width y height:** Ancho y alto de la ventana
- **title:** Título de la ventana
- **GLFWmonitor* monitor:** Monitor donde se crea la ventana.
 - Si nullptr, lo crea en una ventana
 - Si **glfwGetPrimaryMonitor()**, se abre pantalla completa del monitor principal
- **GLFWwindow* share:** Ventana con la que compartir recursos. Normalmente nullptr

void glfwMakeContextCurrent(window): Establece como activo el contexto de una ventana

Contexto: Estructura que almacena todo el estado y los recursos de OpenGL. La ventana es el contexto “mas o menos”

glfwSetWindowTitle(window,title): Cambia el título de una ventana

void glfwGetWindowSize(window, width, height): Obtiene el tamaño de la ventana y lo almacena en width y height

glfwWindowHint(GLFW_RESIZABLE, GLFW_FALSE): Desactiva el escalado

void glfwSwapBuffers(window): Cambia de buffer/Refresca la pantalla

while (!glfwWindowShouldClose(window)): Hace que el bucle corra hasta que se cierre la aplicación

Entrada de Datos

Callback: Pasar una función como parámetro de otra.

- **Función Callback:** Función pasada por parámetros
 - Con la función callback, actualizaremos el estado de los datos de entrada.
- **GLFWkeyfun glfwSetKeyCallback(window, cbfun):** Callback de teclado
- **void foo(window, key, scancode, action, mods):** Función Callback
 - **key:** GLFW_KEY_ESC, GLFW_KEY_SPACE
 - **action:** GLFW_PRESS, GLFW_RELEASE
 - **mods:** GLFW_MOD_SHIFT, GLFW_MOD_CONTROL
- **GLFWcursorposfun glfwSetCursorPosCallback(window,cbfun):** Callback de movimiento del ratón
- **void foo(window, xpos, ypos):** Función de Callback
- **GLFWmousebuttonfun glfwSetMouseButtonCallback(window, cbfun):** Callback de botones del ratón
- **void foo(GLFWwindow* window, int button, int action, int mods):** Función de Callback
 - **button:** GLFW_MOUSE_BUTTON_LEFT
 - **action:** GLFW_PRESS, GLFW_RELEASE
 - **mods:** GLFW_MOD_SHIFT, GLFW_MOD_CONTROL

Pooling: Nos permite consultar el estado del teclado y el ratón en cualquier momento sin necesidad de funciones callback

- El sistema se encarga de guardar todos los cambios del estado de los dispositivos y se actualiza cada frame
- **void glfwPollEvents():** Procesa los eventos si hay alguno pendiente
- **void glfwWaitEvents():** Procesa los eventos o espera hasta que haya
- **int glfwGetKey(window, key):** Polling del teclado. Devuelve GLFW_PRESS o GLFW_RELEASE
- **void glfwGetCursorPos(window, xpos, ypos):** Polling de la posición del ratón
- **void glfwSetCursorPos(window, xpos, ypos):** Establece la posición del ratón
- **int glfwGetMouseButton(window, button):** Polling de botones ratón. Devuelve GLFW_PRESS o GLFW_RELEASE

Gestión de Tiempo

Cuando actualizamos los elementos que componen nuestro juego, es importante que la lógica transcurra a la misma velocidad independientemente de la máquina.

Para solucionar esto especificaremos los movimientos en puntos por segundo en base al tiempo transcurrido desde el frame anterior

Se calcula así:

```
newTime = (glfwGetTime());  
deltaTime = newTime - lastTime;  
lastTime = newTime;
```

glfwGetTime(): Permite obtener el tiempo transcurrido

OpenGL

OpenGL tiene 2 elementos básicos: **Texturas y Vertices**

Al definir los vértices de un objeto, se debe indicar cómo se conectan unos con otros para formar caras y el tipo de primitiva.

Estados

Estados: Configuración interna persistente que afecta a cómo se dibujan los objetos.

glEnable/glDisable: Activa/Desactiva el estado

- **GL_BLEND:** Activa la mezcla de valores de color entre los colores a pintar y los ya existentes
- **GL_DEPTH_TEST:** Activa la comprobación de profundidad en el pintado
- **GL_SCISSOR_TEST:** Restringe el pintado del color buffer a la región especificada por glScissor
- **GL_TEXTURE_CUBE_MAP_SEAMLESS:** Suaviza las aristas de los cube maps modificando el modo de mapeado

Buffers

glEnableClientState(GLenum array): Activa un array de datos en GPU

- **Tipos:** GL_VERTEX_ARRAY, GL_COLOR_ARRAY

void glVertexPointer(size, type, stride, pointer): Define dónde están almacenadas las posiciones de los vértices

void glColorPointer(size, type, stride, pointer): Define dónde están almacenados los colores de los vértices.

void glDrawArrays(mode, first, count): Dibuja la geometría especificada por el buffer de vértices enlazado

- **mode:** Tipo de primitiva a dibujar (GL_TRIANGLES por ejemplo)
- **first:** Índice del primer vértice a procesar
- **count:** Numero de vértices a procesar

void glDrawElements(mode, count, type, indices): Dibuja la geometría especificada por el buffer de índices y el buffer de vértices

- **mode:** Tipo de primitiva a dibujar
- **count:** Número de índices contenido en el buffer de índices
- **type:** Formato en que están definidos los índices en el buffer
- **indices:** Cuando trabajamos con VBO, este parámetro debe ser nullptr.

Para dibujar usamos vertex buffer objects(VBO)

VBO: Permiten enviar la información geométrica de los elementos a dibujar a la memoria de la tarjeta de vídeo

void glGenVertexArrays(n, buffers): Genera el identificador de buffer de tipo vértices. Estos ids tienen dos listas de datos, vértices e índice

void glGenBuffers(n, buffers): Genera el número de buffers especificado por el parámetro n, y coloca sus identificadores en el array buffers, que debe tener al menos n elementos

void glDeleteBuffers(n, buffers): Elimina n buffers cuyos identificadores serán leídos del array buffers

void glBindBuffer(target, buffer): Establece el buffer activo

- **GL_ARRAY_BUFFER:** El buffer especificado contiene información de los vértices
- **GL_ELEMENT_ARRAY_BUFFER:** El buffer especificado contiene los índices

void glBufferData(target, size, data, usage): Envía los datos al buffer. Estos datos deben estar contenidos en el array apuntado por data

- **GL_STATIC_DRAW:** Solicita que los datos se carguen en memoria de vídeo.
- **GL_DYNAMIC_DRAW:** Solicita que los datos permanezcan en la memoria principal.
- **GL_STREAM_DRAW:** Los datos no cambiarán a menudo, y no se van a utilizar frecuentemente, con lo que se volcará en memoria de vídeo cuando sea necesario

Pintado

GL_LINES: Cada par de vértices dibujan una línea.

GL_LINE_STRIP: Dibuja una serie de líneas conectadas entre el punto n y el punto n+1.

GL_LINE_LOOP: Dibuja una serie de líneas conectadas entre el punto n y el punto n+1, y conectando el último y el primer punto.

GL_TRIANGLES: Cada tres vértices dibujan un triángulo.

GL_TRIANGLE_STRIP: Dibuja cada triángulo con los dos últimos vértices del triángulo anterior y el siguiente de la lista.

Tema 3 – Técnicas de Representación y Modelado

Introducción

Modelo: Representación de una entidad que nos interesa estudiar o visualizar

- Sus características básicas se pueden describir mediante geometría
- Sustituye a las técnicas tradicionales de construcción de prototipos

Objetivos:

- Permite que se visualice y comprenda el comportamiento de la realidad
- Proporcionar un medio adecuado para realizar experimentos y predecir efectos de modificaciones

Problemas:

- Se modelan cosas muy complejas y tenemos que recurrir a simplificaciones
- Cuando mas se asemeje a la realidad, mejor será el modelo

Modelo Geométrico: Describen objetos con propiedades geométricas inherentes.

- **Distribución Espacial:** La geometría en función de puntos, líneas o volúmenes más complejos
- **Conectividad:** La estructura de la entidad
- **Valores de Datos Específicos:** Aplicación y Propiedades relacionadas con los componentes

Para cada técnica de representación, consideramos:

- Como está representado el sólido. Que primitivas se usan
- Como almacenar la información y que información se almacena
 - Almacenar información redundante aumenta el tamaño del modelo
 - Disponer de información adicional puede acelerar el proceso de análisis

Etapas de Creación de Modelos:

- Edición del modelo
- Almacenamiento
- Utilización para visualizar o simular el objeto

Propiedades que debe cumplir un modelo de representación:

- **Precisión:** Permite representar un objeto como es en la realidad
- **Dominio:** Conjunto de objetos que se pueden representar en un modelo
- **Ausencia de Ambigüedad:** Una representación debe corresponder a un solo solido

- Algunas técnicas pueden ser interpretadas de mas de una manera. Por ejemplo: dirección positiva de la z.
- **Unicidad:** Un solido se codifica de una única forma
- **Validez:** Es imposible la creación de una representación invalida, que no corresponda a un solido
- **Cierre:** Todas las operaciones sobre solidos deben de ser cerradas. La transformación de un sólido valido lleva a otro sólido valido
- **Compacta:** Reducir el espacio de almacenamiento
- **Eficiencia:** Algoritmos eficientes para calcular propiedades físicas

Existen 2 maneras de generar un modelo geométrico para un objeto concreto:

- Edición manual usando un programa de modelado
- Adquisición de datos 3D a partir de un modelo similar
 - **Digitalizador Manual:** Muestra puntos de la superficie desplazando el digitalizador
 - **Digitalizador Laser:** Emisión de luz estructurada y la adquisición de imágenes desde un punto de vista diferentes

Técnicas de Representación de Objetos

Modelos de Alambre: La información se representa como un conjunto de líneas que representan las aristas que forma dicho objeto

- **Ventaja:**
 - Sencillez
- **Problemas:**
 - Ambigüedad en la representación en pantalla.
 - No permite realizar operaciones booleanas,
 - Incapacidad de representar perfiles curvados.
 - Incapacidad de detectar interferencias entre componentes.
 - Dificultades en el cálculo de las propiedades físicas de los objetos.
 - Incapacidad para aplicar métodos de iluminación y sombreado.

Generación de Sólidos por Primitivas: Define un conjunto de formas solidas tridimensionales relevantes para el área de aplicación.

- Se parametrizan en función de una serie de propiedades
- **Primitiva Parametrizada:** Definición de una familia cuyos miembros varían en unos cuantos parámetros
- Usada en objetos de complejidad alta como combinaciones booleanas de otros objetos más sencillos

Representaciones de Desplazamiento: Al desplazar un objeto a lo largo de una trayectoria por el espacio se barre una región que define un objeto nuevo

- **Desplazamiento Traslacional:** Área bidimensional desplazada a lo largo de una trayectoria para crear un volumen
- **Desplazamiento Rotacional:** Se definen mediante la rotación de un área con respecto a un eje
- **Barridos Generales:** Desplazamientos donde el área o volumen cambia de tamaño, forma u orientación y siguen una trayectoria curva

Representación de fronteras (B-rep): Definen un objeto en función de sus fronteras superficiales, es decir de las caras y superficies

- Pueden basarse en caras planas o curvas
- Polígonos convexos, Triángulos
- Definición en función de sus vértices, aristas y caras
- **Representaciones Superficiales:** Limitadas a fronteras poligonales planas
 - Puede generar representaciones no validas
 - Soportan solamente representaciones de tipo 2-variedad
 - Permiten acelerar determinadas operaciones
- **Representaciones Volumétricas:** Malla de tetraedros
- **Poliedros:** Sólidos acotados por polígonos
 - **Poliedro Simple:** Podemos deformarlo para formar una esfera
 - Cada vértice conecta 2 aristas
 - A cada vértice llegan al menos 3 aristas
 - Las caras no pueden interpenetrarse
- La representación mas simple es una lista de caras cada una representa por una lista de vértices
- Para evitar duplicación de vértices, cada vértice se representa con un índice
- Las caras se representan como un conjunto de vértices adyacentes
- **Tiras y Abanicos:** Cadenas de triángulos que comparten vértices

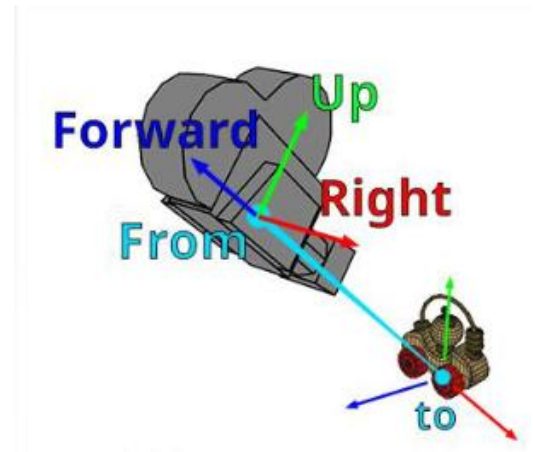
Cámara

La cámara se define por los siguientes datos:

- Posición de la cámara
- Orientación de la cámara
- Punto al que mira

Vectores Necesarios:

- **Forward:** Punto al que mira – Posición de la cámara
- **Right:** Perpendicular a los vectores.
 - $\text{cross}(\text{Forward}, \text{Orientation})$
- **Up:** Perpendicular a los vectores.
 - $\text{cross}(\text{Right}, \text{Forward})$
- **Position:** Transformada de la posición de la cámara.
 - $\text{Vector}(-\text{dot}(\text{right}, \text{pos}), -\text{dot}(\text{up}, \text{pos}), \text{dot}(\text{forward}, \text{pos}))$



Proyección: Para poder usar una escala definida por el usuario, es necesario definir un volumen de renderizado

- Se debe recolocar cada vértice dentro del nuevo sistema
- **Operaciones que realizar:**
 - Trasladar los objetos al origen de coordenadas del nuevo sistema
 - Reescalar coordenadas X, Y respecto al ángulo de visión y "Aspect ratio"
 - Transformar las coordenadas "absolutas" en coordenadas de ventana
- Para la matriz proyección, los plano izquierda/derecha se calcula con el ángulo de apertura de la cámara

Matriz Vista:

$1 / \text{aspect} * \tan(\text{fovy} * 0.5)$	0	0	0
0	$1 / \tan(\text{fovy} * 0.5)$	0	0
0	0	$-(z_{\text{Far}} + z_{\text{Near}}) / (z_{\text{Far}} - z_{\text{Near}})$	$2 * z_{\text{Far}} * z_{\text{Near}} / (z_{\text{Far}} - z_{\text{Near}})$
0	0	-1	1

Matriz Proyección En Perspectiva:

$1 / \text{aspect} * \tan(\text{fovy} * 0.5)$	0	0	0
0	$1 / \tan(\text{fovy} * 0.5)$	0	0
0	0	$-(z_{\text{Far}} + z_{\text{Near}}) / (z_{\text{Far}} - z_{\text{Near}})$	$2 * z_{\text{Far}} * z_{\text{Near}} / (z_{\text{Far}} - z_{\text{Near}})$
0	0	-1	1

Tema 4 – Shaders y Programación en GPU

Shaders

Shader: Programa compilable que se ejecuta en GPU

- Permite hacer programables tareas de las que anteriormente se encargaba el pipeline fijo de la API de renderizado
- Podemos añadir programas al cauce grafico que reescriben como se tratan los datos de entrada

Vertex Shader: Permite programar el proceso de transformación de los vértices

Geometry Shader: Se encargan del ensamblado de fragmentos permitiendo definir nuevas primitivas

Fragment Shader: Se encarga del proceso de coloreado y texturizado de fragmentos

Toda serie de shaders que definen un proceso de renderizado completo forman un programa

Para hacer un shader:

- Se debe escribir un programa y compilarlo usando la interfaz del driver
- Se debe especificar versiones de shader soportadas
- Se deben definir variables de entrada y salida del programa:
 - **Shader de vértices:** Entran y salen vértices
 - **Shader de fragmentos:** Entran “píxeles potenciales” y salen colores
- Un programa esta formado por uno o varios shaders de distinto tipo
 - Los shaders se compilan por separado y se linkan para formar el programa final

GLuint glCreateProgram(): Crea el programa en GPU a partir de los shaders. Devuelve el identificador del programa

void glDeleteProgram(GLuint program): Elimina el programa pasado por parámetros

void glAttachShader(GLuint program, GLuint shader): Añade un shader al programa especificado

void glLinkProgram(GLuint program): Enlaza todos shaders de un programa para crear el programa ejecutable

void glUseProgram(GLuint program): Reemplaza las tareas del pipeline de renderizado de OpenGL por las definidas en el programa.

GLuint glCreateShader(GLenum shaderType): Crea un nuevo shader

- **Tipos:**
 - **Shader de Vertices:** GL_VERTEX_SHADER
 - **Shader de Fragmentos:** GL_FRAGMENT_SHADER

void glShaderSource(shader, count, string, length): Asigna el código fuente de un shader

void glCompileShader(GLuint shader): Compila el código fuente del shader asignado anteriormente

void glDeleteShader(GLuint shader): Elimina un shader. Una vez que un shader ha sido enlazado en un programa, podemos eliminarlo

Lectura de Código del Shader:

```
string code;
std::ifstream f(file);
if (f.is_open()) {
    code = std::string(std::istreambuf_iterator<char>(f), {});
}
else {
    std::cout << "ERROR: FICHERO NO ENCONTRADO " <<
        __FILE__ << ": " << __LINE__ << " " << file << "\n";
}
```

Creación del Programa:

```
programID = glCreateProgram();

int vertexShaderID = glCreateShader(GL_VERTEX_SHADER);
glShaderSource(vertexShaderID, 1, &vertexCode, nullptr);
glCompileShader(vertexShaderID);

int fragmentShaderID = glCreateShader(GL_FRAGMENT_SHADER);
glShaderSource(fragmentShaderID, 1, &fragmentCode, nullptr);
glCompileShader(fragmentShaderID);

glAttachShader(programID, vertexShaderID);
glAttachShader(programID, fragmentShaderID);

glLinkProgram(programID);
```

Errores de Compilación:

```
GLuint retCode;
char errorLog[1024];
GLuint fragment_compiled;
glGetShaderiv(shID, GL_COMPILE_STATUS, &fragment_compiled);
if (fragment_compiled != GL_TRUE)
{
    GLsizei log_length = 0;
    GLchar message[1024];
    glGetShaderInfoLog(shID, 1024, &log_length, message);
    std::cout << "ERROR " << fileName << "\n" << message << "\n\n";
}
```

Errores de Linkado:

```
GLuint program_linked;
glGetProgramiv(prID, GL_LINK_STATUS, &program_linked);
if (program_linked != GL_TRUE)
{
    GLsizei log_length = 0;
    GLchar message[1024];
    glGetProgramInfoLog(prID, 1024, &log_length, message);
    std::cout << "ERROR "\n" << message << "\n\n";
}
```

Shader de Vértices:

- Se recibe cada vértice en el espacio del modelo y ha de devolverse en el espacio de proyección.
- Se ejecuta una vez por vértice
- Para realizar la transformación se utiliza la matriz MVP
- Se pasa también la matriz modelo para trabajar con iluminación
- **gl_Position:** Variable predefinida que define la posición final de un vértice
- **attribute:** Indica que una variable que define datos de vértice y cuyo valor se lee del buffer de vértices enlazado
- **uniform:** Variables usadas como parámetro extra, copiados a cada instancia del vertex shader

Los programas deben tener inicializadas sus variables globales.

- Se deben conseguir identificadores numéricos que sirven para indicar que variables se quiere modificar en cada momento

GLint glGetUniformLocation(program, name): Devuelve la localización de la variable llamada name del vertex shader indicado. Si no se ha encontrado, devuelve -1.

layout(location=2) attribute vec3 pos: Se puede forzar la localización de la variable

Las variables se inicializan en CPU así que deberemos conocer sus nombres y su índice en el programa

Se puede interrogar al programa compilado:

- Obtener el número de variables de tipo uniform o atributos:
 - glGetProgramiv(programID, GL_ACTIVE_ATTRIBUTES, &count);
 - glGetProgramiv(programID, GL_ACTIVE_UNIFORMS, &count);
- Obtener las características de esas variables:
 - glGetActiveAttrib(programID, (GLuint)idx, bufSize, &length, &size, &type, name);
 - glGetActiveUniform(programID, (GLuint)idx, bufSize, &length, &size, &type, name);
- Obtener el índice de una variable en el shader compilado:
 - glGetUniformLocation(programID, varName)
 - glGetUniformLocation(programID, varName)

void glEnableVertexAttribArray(GLuint index): Activa la escritura en una variable attribute cuya localización se pasa en el parámetro index

void glVertexAttribPointer(index, size, type, normalized, stride, pointer): Especifica cómo leer del buffer de vértices los datos de una variable attribute

- **index:** Localización/identificador de la variable.
- **size:** Número de componentes a escribir por vértice (4 para posiciones, normales, colores, 2 para coordenadas de textura).
- **type:** Tipo con el que le pasamos los datos (por ejemplo, GL_FLOAT, GL_UNSIGNED_BYTE...).
- **normalized:** Indica si se deben normalizar los datos enviados. Por ejemplo, si definimos los colores con valores [0, 255], se normalizarán automáticamente al rango [0, 1] si este parámetro vale true.
- **stride:** Tamaño del paquete de datos completo para un vértice (el sizeof de la estructura donde guardamos los datos de un vértice).
- **pointer:** Offset dentro del buffer activo en que se encuentra el primer dato (es un entero que debemos castear a void*).

Shader de Fragmentos:

- Recibirá fragmentos (píxeles potenciales) que forman cada una de las primitivas pasamos al proceso de renderizado
- Se ejecuta una vez por fragmento
- Dependiendo del efecto gráfico, puede ser necesario pasar la información desde el shader de vértices
 - **attribute:** Valores que se recibirán una vez por cada vértice
 - **out/in:** Cuando es de salida en el shader de vértices y de entrada en el de fragmentos
- Como el shader de vértices se ejecuta una vez por vértice y el de fragmentos una vez por fragmento, para calcular el valor que recibe el fragmento se realiza interpolación de los datos
- **gl_FragColor:** Guarda el color final del fragmento

Vertex Shader:

```
#version 330

uniform mat4 MVP;
attribute vec4 vPos;
attribute vec4 vColor;
out vec4 fColor;

void main() {
    gl_Position=MVP*vPos;
    fColor = vColor;
}
```

Fragment Shader:

```
#version 330

in vec4 fColor;

void main() {
    gl_FragColor = fColor;
}
```

Variables Uniform: Variables que no varían durante toda la ejecución de los shaders

GLint glGetUniformLocation(program, name): Devuelve el localizador de una variable uniform

Funciones de Establecimiento de Datos:

- void glUniform1i(GLint location, GLint v0)
- void glUniform1f(GLint location, GLfloat v0)
- void glUniform2f(GLint location, GLfloat v0, GLfloat v1)
- void glUniform3f(GLint location, GLfloat v0, GLfloat v1, GLfloat v2)
- void glUniform4f(GLint location, GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- void glUniformMatrix4fv(GLint location, GLsizei count, GLboolean transpose, const GLfloat*value)

Cuando vayamos a dibujar un objeto, hay que dibujar sus buffer objects para enlazarlos con la entrada de parámetros de nuestro shader

1. **glBindVertexArray(array)**
2. **glBindBuffer(GL_ARRAY_BUFFER, vertexId)**
3. **glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, idxId);**
4. **Activamos el shader que queremos usar:** `glUseProgram(shaderID);`
5. **Seteamos parámetros de tipo Attribute y Uniform:**
 - **Atributos:**
 - `glEnableVertexAttribArray(location);`
 - `glVertexAttribPointer(.....,offsetof(estructura,registro))`
 - **Uniform:**
 - `glUniform1i(location, value);`
 - `glUniformMatrix4fv(location, 1, GL_FALSE, glm::mat4);`
6. **Dibujar la malla:** `glDrawElements(GL_TRIANGLES, numberOfIndexes, GL_UNSIGNED_INT, nullptr);`

Tema 5 – Iluminación y Texturas

Iluminación

Idealmente un emisor de rayos nos ilumina los objetos.

Dependiendo del ángulo de incidencia, el color percibido varia.

Los objetos también pueden tener distintas propiedades para sus superficies por lo que la misma luz tendrá distintos resultados sobre el objeto.

Color:

- La luz se puede representar como una onda.
- Dependiendo de la longitud de onda, podemos definir un color u otro
- La mezcla de ondas produce que varíen los colores.
 - Propiedades aditivas permiten que dos emisores de distintos colores puedan mezclar en un momento sus haces de luz para dar un tercer color

Tipos de Emisores:

- **Puntos de Luz:**
 - Emite luz en todas las direcciones
 - No tienen tamaño, solo posición
- **Fuentes de Luz Distribuida:**
 - Emiten luz direccional
 - Tiene tamaño

Tipos de Luz:

- **Ambiental:** Media de toda la luz que recibe el objeto desde el emisor.
 - La luz proviene de todos los puntos del espacio
 - Ilumina por igual todas las facetas del objeto.
 - Se usa como base para iluminar y colorear el objeto.
 - Se entiende como constante
- **Difusa:** Incide en el objeto en todos sus puntos con la misma dirección
 - Tipo de luz direccional
 - Dependiendo del ángulo de incidencia, creará un efecto de sombreado
- **Especular:** Añade brillos al objeto y su intensidad depende una constante del material
 - Tipo de luz direccional
 - Depende del ángulo de visión
- **Emisiva/Emisora:** El emisor de la luz es el objeto
 - Solo se usa modelos de iluminación global
 - A los valores de iluminación hay que sumarles las luces reflejadas
 - Depende de los valores de iluminación del objeto, y de sus propiedades de reflexión

Modelo de Iluminación De Phong

Define la luz como la suma de los valores de la iluminación ambiental, difusa y especular para cada punto del polígono.

No hay luz emisor desde el objeto, únicamente se refleja la luz que viene de los focos o reflejada de otros objetos

$$I_{phong} = I_{amb} + I_{dif} + I_{esp}$$

Luz Ambiental:

- $I_{amb} = I_a \cdot k_a$
- **I_a**: Constante de Iluminación
- **k_a**: Constante del material. Depende de las propiedades ópticas del objeto

Luz Difusa:

- $I_{dif} = I_d \cdot k_d \cos(\theta) = I_d \cdot k_d \cdot (L \cdot N)$
- **N**: Normal de la superficie en un punto
- **L**: Vector de incidencia de la luz
- **I_d**: Intensidad de Fuente de Luz
- **k_d**: Coeficiente empírico de reflexión. Depende de λ . Entre 0 y 1

Luz Especular:

- $I_{esp} = I_s \cdot k_s \cos^n(\Omega) = I_s \cdot k_s \cdot (R \cdot V)^n$
- **R**: vector del rayo de la luz reflejado
- **V**: vector del visor
- **I_s**: intensidad de la fuente de luz
- **k_s**: coeficiente empírico de reflexión especular ($0 \leq k_s \leq 1$)
- **n**: un índice que simula la rugosidad de la superficie ($1 \leq n < \infty$, 1: mate, ∞ : espejo)
- **Ω**: ángulo entre V y R

Cuando la luz viaja por el espacio se va atenuando según una función de atenuación

$$f(d) = \frac{1}{d^2}$$

La atenuación solo afecta a la luz difusa y a la luz especular.

$$I_{phong} = I_{amb} + f(d)(I_{dif} + I_{esp})$$

Color:

- Para poder calcular la iluminación usando colores, es necesario especificar los coeficientes de iluminación para cada componente del color
- Se suelen especificar los coeficientes como vectores de 3 entradas

Como realizar ejercicio:

- **Datos:**
 - Triángulo: $P1 = (1,1,1)$ $P2 = (0,2,1)$ $P3 = (0,0,1)$
 - Constantes de luz: $k_a = 0.7$ $k_d = 0.9$ $k_s = 0.6$ $n = 10$
- Punto a iluminar: $(0.33,1,1)$
- Intensidad de luz ambiente (I_a) = 0.1
- Punto de luz blanco:
 - Posición = $(1,1,5)$
 - Intensidad especular y difusa (I_s, I_d) = 0.5
- Posición del visor = $(1,2,5)$

- **Paso 1:** Sacar iluminación ambiental
- **Paso 2:** Sacar iluminación difusa
 - **Calcular Normal al Triángulo (N):**
 - Encontrar 2 vectores pertenecientes al triángulo. Vectores entre 2 puntos
 - Producto Vectorial de ambos vectores y normalizar el vector
 - **Calcular el Vector de Incidencia (L):**
 - Punto de la luz – Punto a iluminar
 - Normalizar
- **Paso 3:** Sacar iluminación especular
 - **Calcular R:** $R = 2N(N \cdot L) - L$
 - **Calcular V:** Punto Visor – Punto a Iluminar
 - Si $R \cdot V$ es menor que 0 $\rightarrow$ Redondear a 0
- **Paso 4:** Sacar iluminación total

Sombreado

Hay 2 fases para la iluminación:

- Iluminación de vértices (procesador de vértices)
- Sombreado (procesador de fragmentos)

La iluminación se realiza sobre los vértices y luego se interpola a lo largo del polígono

Etapas de Sombreado: Cálculo de interpolaciones en cada punto del polígono

Tipos de Sombreado:

- **Sombreado Plano:** Se calcula el valor de la iluminación sobre un único punto del polígono
 - Se usa el mismo valor para todos los puntos de este
 - Rápido, pero poco preciso
- **Sombreado de Gouroud:** Se calcula la iluminación en cada uno de los vértices
 - Se calcula la normal del vértice como la suma de las normales de los triángulos que lo comparten
 - Calcular la intensidad en los puntos interiores como una interpolación lineal de los valores de los vértices
- **Sombreado de Phong:**
 - Calcular normal en los vértices. Igual que antes, la normal se calcula como la media de las normales de los polígonos adyacentes.
 - Calcular la normal de los puntos interiores por interpolación lineal de las de los vértices.
 - Calcular la intensidad en cada punto mediante el modelo de iluminación de Phong. Ahora se puede introducir también la reflexión especular

Shaders

Se necesita guardar la información de las normales por vértice

- **Sombreado Plano:** Sólo tiene sentido para vértices no compartidos entre caras
- **Sombreado Smooth:** Simula una superficie "curva" con pocos polígonos
 - Los vértices deben estar compartidos entre caras adyacentes

Shader de Vértices:

```
uniform mat4 MVP;
uniform mat4 M;

attribute vec4 pos;
attribute vec4 color;
attribute vec4 normal;

out vec4 fcolor;
out vec4 fpos;
out vec4 fnorm;

void main()
{
    mat4 invT=inverse(transpose(M));
    fnormal = (invT*vnormal);
    fpos=M*pos;

    gl_Position=MVP*pos;
    fcolor=color;
}
```

Shader de Fragmentos:

```
uniform vec3 lightPos;
uniform vec3 eyePos;

in vec4 fpos;
in vec4 fnormal;

.....
vec3 N=fnormal;
vec3 L=lightPos-fpos;
diffuse=max( dot(L,N),0.0f);

float specular=0.0f;
vec3 R=normalize(reflect(L,N));
vec3 EYE=normalize(fpos.xyz-eyePos);
specular=pow(max(dot(R,EYE),0.0f),shininess);

light=((ambient+diffuse*contribDiff)+specular*contribSpec);
```

Texturas

Tenemos una capacidad limitada para generar superficies complejas. Las imágenes nos dan la ilusión de geometría.

Texture Mapping: Pintado de imágenes sobre polígonos

Texture Map: Imagen en 2 dimensiones

Texels: Mediante coordenadas de texturas

Cada píxel se sustituye por un texel.

Hacemos corresponder una coordenada (u,v) al espacio de coordenadas del polígono (s,t):

- Se da una coordenada de textura para cada vértice
- Interpolamos la conversión para los puntos entre vértices

Cada objeto tiene coordenadas de objeto y de textura:

- **Coordenadas de Objeto:** Sus vértices. Donde está en el espacio
- **Coordenadas de Textura:** Posición de un vértice respecto dentro de la textura.
 - En rango 0-1
 - Se interpolan entre vértices

Mapas de Profundidad: Utilizan las texturas para modificar la geometría de las superficies

- Utiliza el valor del texel para modificar el valor de la normal en cada píxel
- **Heightfield:** Campo de altura definido por los texels
 - Permite modelar superficies rugosas

Bump Mapping: Técnica para mapear bultos en polígonos

- Usa el mapa de normales para indicar el valor de la normal de cada píxel
- En un polígono todos los puntos deberían tener la misma normal
 - La textura guarda información sobre las normales pero no del color o la imagen
- Añade detalle a modelos con pocos polígonos

Displacement Mapping: Alteramos la geometría para no generar normales que son inconsistentes con la geometría real (Los bumpmaps hacen esto).

Minmaps: Define varias texturas a diferentes resoluciones y las almacena. Se almacena una por cada nivel

- Podemos tener objetos unos cerca de otros
- Los objetos mas cercanos se han de ver con mas detalles que los lejanos

Mezclado de Color

Blending: Técnica que consiste en mezclar los colores del pixel que vamos a pintar con los colores del píxel existente en el color buffer en esas coordenadas

- El píxel para pintar tiene valores RGBA
- El color buffer tiene valores RGB
- Según la forma de mezclado obtendremos distintas opciones de pintado

Formas de Mezclado:

- **SOLID:** Los píxeles a pintar reemplazan los píxeles del back buffer ignorando Alpha
- **ALPHA:** Los píxeles a pintar se mezclan con los del back buffer considerando el valor alpha de origen
- **MULTIPLICATIVE/MODULATIVE:** El color a pintar se multiplica por el color del back buffer
 - Produce efectos de oscurecimiento
- **ADDITIVE:** El color a pintar se suma al color del back buffer
 - Produce efectos de iluminación

Estado GL_BLEND: Debe de estar activado para poder mezclar colores

void glBlendFunc(GLenum src_factor, GLenum dst_factor): Establece el modo de pintado

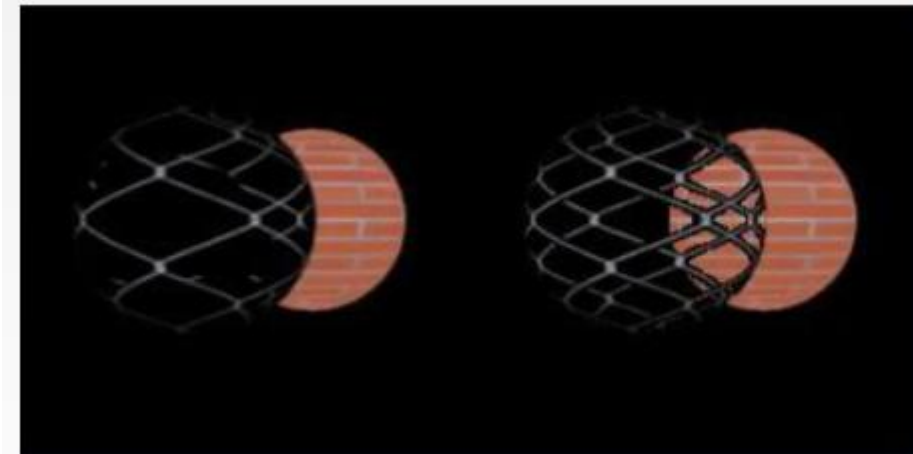
- **Formula de Pintado:** $\text{src_factor} * \text{src_color} + \text{dst_factor} * \text{dst_color}$
 - **SRC_COLOR:** Color del objeto superior
 - **DST_COLOR:** Color del objeto inferior
- **Valores de src_factor y dst_factor:**
 - GL_ZERO
 - GL_ONE
 - GL_SRC_COLOR
 - GL_DST_COLOR
 - GL_ONE_MINUS_SRC_COLOR
 - GL_ONE_MINUS_DST_COLOR
 - GL_SRC_ALPHA
 - GL_ONE_MINUS_SRC_ALPHA

Orden de Pintado

Hay que distinguir entre aquellos objetos en los que consideramos que todos sus píxeles son parcial o totalmente transparentes y los que son opacos

En ambos casos, se lee la información previamente en el depth buffer para no pintar objetos que se encuentren detrás de los sólidos

Los elementos transparentes se suelen pintar después que los sólidos



- Para el primer tipo de transparencia, no escribimos en el depth buffer
- Para el otro tipo de transparencia, escribimos en el depth buffer si el texel es opaco, y no lo hacemos si el texel es transparente
- **glDepthMask(GL_TRUE):** Activa la escritura en el depth buffer
- **glDepthMask(GL_FALSE):** Desactiva la escritura en el depth buffer
- Para hacerlo a nivel de fragmento, utilizaremos la sentencia discard en el fragment shader

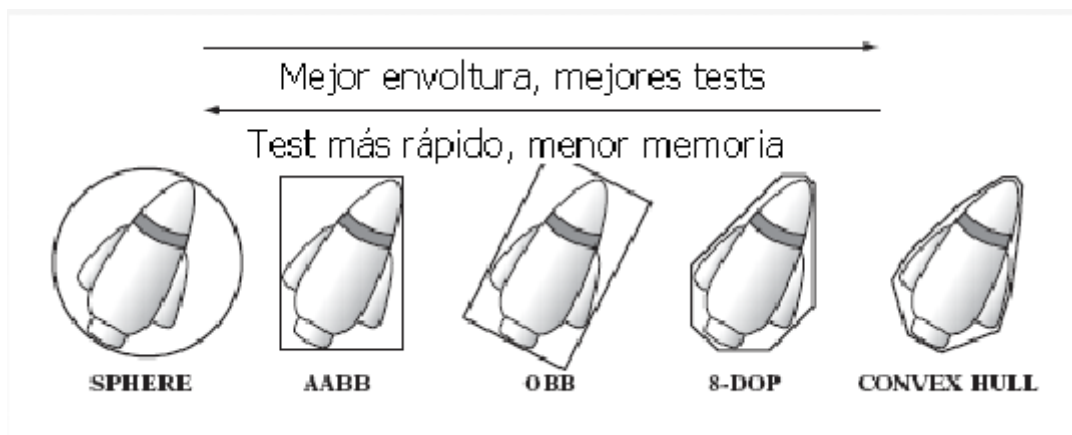
Tema 6 – Colisiones

Volúmenes Envolventes

Propiedades para elegir volúmenes envolventes óptimos:

- Test de intersección muy rápidos
- Entorno “ajustado” al objeto
- Rápidos de crear/calcular
- Fáciles de transformar y rotar
- Coste bajo de memoria

Se necesita buscar el equilibrio entre estas características:



Esferas

Lo más sencillo de construir:

Ventajas:

- Rápido de construir
- Bajo consumo de memoria
- Test de intersección muy rápidos

Desventajas:

- Test poco precisos

Axis Aligned Bounding Boxes (AABB)

Exaedros que tienen las caras alineadas en los ejes x,y,z

Ventajas:

- Mejora la precisión de los tests
- Fácil de construir
- Consume poca memoria

Desventajas:

- No se ajustan todos los casos igual
- Si el objeto cambia de forma u orientación, hay que recalcular

Oriented Bounding Boxes (OBB)

Los exaedros ahora no tienen que estar alineados

Ventajas:

- No hay necesidad de recalcular
- El mejor ajuste posible en todos los casos con cajas

Desventajas:

- Mas difícil de representar → Mas memoria
- Tests más complejos

Discrete-Orientation Polytopes

Poliedros cuyas caras son paralelas 2 a 2. Se pueden construir a partir de cajas OBB más pequeñas

Ventajas:

- Test más precisos
- Poco incremento de memoria

Desventajas:

- Tests más complejos
- Construcción más compleja

Otros

Cilindros, conos, elipsoides, pirámides, otros poliedros... En general cualquier otro tipo de volumen envolvente tendrá un coste de creación o de realización de tests mucho más altos

Jerarquía de Volúmenes Envolventes

Cuando se tiene un escenario con varios objetos que podrían estar colisionando, establecemos una jerarquía para los tests

Cada BV este envuelto en otros volúmenes envolventes mayores.

Árboles

La jerarquía se presenta en forma de árbol de forma que los intermedios son BV contenedores y los nodos hoja contienen los objetos únicamente

Características:

- Cada nodo de la jerarquía debe contener el mínimo volumen
- Igualmente, la suma de todos los volúmenes debe ser mínima
- Se debe de tener mucho cuidado con los nodos más cercanos a la raíz
- El volumen de solapamiento de nodos hermanos debe ser mínima
- La jerarquía debe estar balanceada con respecto a los nodos y su contenido

Construcción del Árbol:

- Los nodos padres son celdas de grano grueso, y cuelgan “n” hijos
- Los nodos hijo son el resultado de dividir el nodo padre en “n” divisiones iguales.
- **Top-Down:** Se parte de un volumen envolvente que contenga a todos los objetos, y se va dividiendo en volúmenes más pequeños, a la vez que se divide el árbol.
 - **Ventajas:**
 - Rápido
 - Fácil de Implementar
 - **Desventajas:**
 - No genera los mejores arboles
- **Bottom-Up:** Se parte de los volúmenes envolventes más básicos (los objetos), y se van construyendo volúmenes envolventes más grandes a partir de los más cercanos.
 - **Ventajas:**
 - Mejores Arboles
 - **Desventajas:**
 - Mas difícil de implementar
 - Mayor Tiempo de Computo
- **Inserción:** Se empieza por un árbol vacío, y se van insertando los nodos de forma que se mantenga el equilibrio entre las ramas.
 - **Ventajas:**
 - Rápido como top-down
 - Mejores resultados que bottom up

BSP-Trees: Son árboles binarios que representan las particiones del escenario

- Las particiones se realizan a partir de divisiones arbitrarias.
- Este tipo de árboles no se suelen usar para simulaciones con objetos en movimiento, ya que habría que reconstruir el árbol en cada paso y es algo costoso
- Se suelen usar para objetos que se encuentren en background y que rara vez se moverán

Spacial Partition

Este sistema se basa en la división del espacio en celdas de tamaños similares para después analizar los polígonos que caen dentro.

Problema: Saber cual es el tamaño de celda apropiado para dar buenos resultados

Lo ideal seria una división cercana al tamaño de los elementos

Grids Jerárquicos

Para solucionar el problema de la partición espacial, se puede recurrir a una jerarquía de grids.

De esta forma se empieza a particionar el espacio con grids de grano grueso, y en el momento que se necesite se subdivide una celda en celdas más pequeña

A la hora de realizar los test de colisiones, se iría directamente a las subceldas más pequeñas